Policy Delegation for the ZPR Reference Implementation

Mathias Kolehmainen, Michael Dubno, Jan 26, 2026
(Updated Jul 20, 2026)

1. RFC-19 Policy Delegation for the ZPR Reference Implementation

Delegation is the act of assigning authority, responsibility, and specific tasks to another person or group, typically with defined constraints. This document briefly introduces the problems that delegation solves, and then describes the solution we have implemented in the Reference Implementation.

1.1. Why Delegation?

A non-trivial ZPR deployment relies on several categories of organizational data and configuration:

- Directory services (AD/LDAP)
- User, group, and attribute management
- Service and policy management
- Credential management
- Physical and virtual infrastructure management
- Physical and ZPR network management

Responsibility for these systems is typically distributed across different organizational teams. Each system implements its own mechanisms for authorization and delegation. For example, Active Directory, databases, and enterprise applications define who may create, read, modify, and delete information, and who may delegate those permissions to others.

These systems act as trusted sources for ZPR. Authentication, attribute management, infrastructure management, and ZPR deployment continue to be administered using existing tools and organizational hierarchies.

> **ZPR/ZPL does not directly provide authentication, reference data, or delegation management. Instead, it relies on trusted sources. ZPR enforces network policy using information obtained from those trusted sources.**

In addition to allowing policy to be compartmentalized along organizational lines, delegation makes a large policy easier to manage because each author is responsible for only a small portion closely related to their infrastructure.

2. Definitions

The following terms are used throughout this document.

Assertion: A declarative ZPL statement of policy intent that is used to verify whether the provided policy correctly permits or denies communication. that should not be permitted.

Domain (Policy Domain): The unit of policy delegation in the ZPR reference implementation. A domain combines a service namespace, credentials for trusted services, restrictions on permitted policy, and a ZPL policy. A domain is restricted to owns service names under its DNS root, and policy in that domain can define and allow access only to services in that domain.

Domain envelope: The portion of a domain controlled by the delegator. The envelope includes the namespace, trusted service bindings, credentials, restrictions, and delegation metadata that constrain the domain contents.

Identity: A key used to look up attributes associated with an endpoint, user, device, or service. ZPR uses authenticated identities to retrieve attributes and evaluate policy.

Namespace: A context in which names are defined. It must be hierarchical. In this document, a delegated namespace is usually represented as a DNS suffix , such as `.marketing.corp.com`, under which a domain may define service names.

Root domain: The top-level domain in a delegated ZPR deployment. The root domain owns the base namespace and may delegate portions of that namespace to child domains.

Service: An application that sends or receives packets and has a name, an identity and attributes. In delegated policy, services are the protected objects defined inside a domain namespace.

Service discovery: The process of resolving a service name into the protocol details and network address information needed to reach the service. In this document, service discovery is treated as a policy-controlled operation.

Service name: A name used to locate a service. In delegated domains, service names are interpreted within the domain namespace and resolved to fully qualified DNS names under the domain's DNS suffix.

Trusted service: A trusted source of authentication or attributes. These are declared or referenced in ZPL and configured and accessed with credentials through a policy domain.

ZPL: Zero-trust Policy Language. ZPL is the human-readable language used to define, audit, and enforce communication policy in a ZPRnet.

ZPRnet: A ZPR network or group of interconnected ZPR nodes that enforce communication policy using visas, compliant flows, and ZPL rules.

2.1. Problem Space

Delegation in a ZPR network introduces challenges in three related areas: service definition, policy management, and service discovery.

2.1.1. Services

Services are the objects being delegated and protected. A delegation model must:

- Declare services within delegated administrative namespaces.
- Prevent service name collisions between administrative domains.
- Prevent unauthorized parties from claiming or redefining existing services.
- Ensure that access policy remains tightly coupled to the service definition.
- Provide a mechanism for service discovery.
- Optionally restrict service discovery so that unauthorized actors cannot determine whether a service exists.

2.1.2. Policies

Policies express delegated authority over services. A delegation model must:

- Ensure that policy authors can only define policy within their delegated scope.
- Prevent policy authors from affecting services outside their delegated namespace.
- Support restricted access to trusted services and attribute sources.
- Prevent policy definitions from becoming a source of unauthorized information disclosure.
- Define a deterministic priority model when multiple delegated policies apply.

2.1.3. Service Definition

Services are given names in policy and are typically located by their name rather than by address. A delegation model must therefore integrate with existing naming and discovery systems.

A delegation model should:

- Support the resolution of service names into network addresses.
- Allow service visibility to vary according to policy.
- Permit different actors to receive different service endpoints for the same service.
- Permit service endpoints to vary between sessions when required.
- Treat service discovery itself as a policy-controlled operation.

2.2. Solution

To support policy delegation we added several capabilities to the Reference Implementation Visa Service:

1. Resolution of service names to addresses is fully under ZPR control. The ZPRnet provides DNS for services hosted in ZPR.

   This gives the visa service the ability to control not just who can place a service within a DNS domain, but also who can find it. The ZPR administrator can also make use of DNS `cname` records to reorganize the policy delegation hierarchy in arbitrary ways. The Visa Service has a service lookup API which is accessed by CoreDNS via a plugin.

2. A policy management system (PMS) for creating, updating, reading and deleting policy tied to policy *domains*.

   The Visa Service must enforce a collection of policies that are each constrained to parts of the corporate namespace. As a content management system it enforces permissioned user access. Interaction with the PMS is through a REST API and access is controlled by API keys and/or access tokens. The PMS also allows for a domain to be sub-delegated in a way that maps naturally to how domain names are used. For example, the domain tied to `marketing.corp.com` could delegate `accounts.marketing.corp.com` to another administrator.

3. Policy domains use their own credentials to interact with trusted services.

   To fit in with access control on existing trusted services (eg, attribute databases, LDAP, etc) each domain is given credentials by the delegator that permit it domain-appropriate trusted service access.

4. Within a domain, policy is subject to restrictions set by whoever configured the domain – its **delegator**.

   When a policy domain is created, its creator can use a subset of ZPL and assertions to restrict the kinds of policy rules that may be used within the domain. These restrictions cannot be changed from within the domain. A domain in a hierarchy of delegated domains is subject to the restrictions imposed by every domain above it in the hierarchy.

5. The Visa Service handles compilation of ZPL.

   The Visa Service needs to enforce ZPL restrictions that are authored separately from a domain policy. To make this work the Visa Service is responsible for actually compiling the domain policy, which it does in the presence of all applicable restrictions. Only policies that pass compilation can be applied to the network.

2.3. Policy Domains

ZPR uses the concept of a **domain** to describe the unit of delegation of a ZPR policy. A domain incorporates:

1. The namespace for all services.
2. Credentials for accessing reference data through trusted services.
3. Restrictions on what is allowed to be expressed in the domain policy.
4. A policy written in ZPL.

The first three items in a domain are managed by the domain creator (aka *delegator*), and we can think of these as comprising the domain "envelope". The final item, the policy, can be thought of as the "contents" of the "envelope". The "contents", written in ZPL by the *delegatee*, declares services and their associated policies.

Each domain's namespace is defined by a DNS suffix (a partial name such as .marketing.corp.com) under which all of its services can be found. Attributes from trusted services match services to their providers, and at runtime a request to a service is matched by address, protocol and port.

ZPL (policy) always exists in a domain. A simple ZPRnet installation has a single, unnamed domain; explicit domain naming is only required when you want to use delegation.

Within a domain, policy statements can only reference attributes accessible via the domain's credentials, and can only affect services declared within the domain's namespace. For example:

```
Allow interns to access lifecycle:test services
```

In the above `services` means "services in this domain" such as "webservice.marketing.corp.com".

2.4. Configuring a domain

To delegate policy a top level ZPR administrator first needs to organize the service namespace. This step is tightly coupled to service discovery for which we rely on DNS. For example, if the root namespace is "corp.com" the administrator may split the namespace into "marketing.corp.com" and "finance.corp.com" with the intention of delegating those service areas to separate groups within the organization. What this means in practice is that the marketing department is free to declare services with names like `database.marketing.corp.com` or

`addserver.marketing.corp.com`, while the finance department can declare services with names like `database.finance.corp.com`, etc.

Next the administrator decides how each domain will access the reference data available on the network. Reference data, such as attributes, is accessed through *trusted services*, for example an LDAP service. Access to those services requires credentials which must be specified for each domain. Attributes available to a user with a finance role may be different from those available to a user with a marketing role. This is an organizational IT decision not managed by ZPR, but the support for domain credentials means ZPR adheres to organizational policies.

Finally the administrator adds restrictions to each delegated domain. The restrictions are written in a subset of ZPL: only `never allow` statements and assertions are permitted. As an example, the administrator may include a statement such as:

```
Never allow role:finance services to access internet-gateway services.
```

This prevents finance services from ever communicating with the public internet, regardless of what the domain policy permits.

Note that the restrictions on a domain run under the credential of the domain creator – not the delegatee.

The key consequence of domain delegation is that the delegator always retains the ability to set restrictions for a delegated namespace. A delegatee cannot set policy for anything outside the namespace defined by its delegator.

Since domains set service namespaces, and services have names, it is best practice to leave the assignment of addresses to the ZPRnet itself. That way there can be no accidental duplicate address assignment. However, even if IP addresses are set in the policy, duplicate address assignment can be caught by the visa service at policy install time.

Within a domain a policy can include `allow` and `never allow` statements that control access to services declared in the domain. The policy can also include `never allow` statements that restrict access *from* services declared in the domain *to* other services regardless of their domain. Note that a domain policy can never include `allow` statements that control access to services declared outside of the domain.

3. Nested Delegation

The policy domain system is flexible enough to support nested delegation to arbitrary levels: a delegatee can be permitted to further delegate their namespace to others. For example, an admin in charge of `marketing.corp.com` can delegate `it.marketing.corp.com` or any other subordinate domain (in the DNS sense) as she sees fit.

Restrictions imposed by any ancestor domain are enforced at all levels; a deeply nested delegatee cannot circumvent a `never allow` restriction set by any of its predecessors. Unlike restrictions, the actual domain policy only ever applies to the domain it is in.

4. Delegation Attributes

Attributes must be carefully controlled to prevent policy within a domain from matching actors or resources that it should not match. Within a domain, access to attributes can be controlled

through the careful configuration of access credentials, through assertions included in the domain restrictions, or through both mechanisms.

A service declared within a domain may be bound only to an actor whose attributes identify it as a member of that domain. The trusted service that provides the domain attribute, and the site-specific name of that attribute, are defined in the ZPRnet configuration. Every actor that provides a service in the ZPRnet must be assigned a domain attribute before it may provide that service.

When a service is declared, the system checks the domain attribute automatically. The domain attribute does not need to be included manually in the service declaration. If it is included, it may refer only to the current domain.

For example, if the system is configured to use a domain attribute named `domain`, then the following declaration would not be permitted within the `finance` domain:

```
Declare shadow-service as a service with port:443 provided by domain:marketing.
```

This declaration is not permitted because the finance domain may declare services only on actors that belong to the finance domain. The declaration attempts to bind a service to an actor in the marketing domain.

The set of attributes returned for an actor may depend on the credentials used to access the attribute service. For example, policies in the `marketing` domain may use attribute names that are not available to policy writers in the `finance` domain.

Using domain-specific attributes whose availability is controlled by access credentials is the preferred practice. Assertions may also be added to the domain restrictions to prevent a delegated administrator from using specific attributes:

```
Assert that no service uses the attribute marketing-service-role.
```

5. Compiler Responsibilities

The policy compiler is the first enforcement point for delegation. Given a domain and domain configuration, it is responsible for ensuring that the policy only uses attributes that are allowed for that delegated author.

The compiler determines the set of allowed attributes from the trusted services. That set may then be further reduced by the domain restrictions. These constraints are enforced statically at compile time by generating compilation errors, preventing the creation of a binary policy file. This design prevents a delegated administrator from accidentally or intentionally authoring policy that escapes their delegated scope.

Even though the Visa Service manages compilation, the compiler is still available as a standalone tool and can be useful for local testing even without access to the domain restrictions. API calls to the visa service can be used by policy administrators to “test compile” their domain policy in the fuller network policy context.

6. Visa Service Behavior

In a non delegated environment, a visa service processes a single policy in the global domain, and makes decisions based purely on that policy. In a delegated environment, the visa service

processes many domains at once.

A crucial invariant is that `never allow` rules and assertions are enforced everywhere in the delegation hierarchy. When traffic is evaluated, the visa service always denies if a `never allow` statement matches. Policy is evaluated based on the domain holding the service being accessed. In the absence of a `never allow` the first `allow` statement in the policy matches.

Each domain maintains its own attribute cache for the trusted services it uses. This keeps attributes in domains isolated from one another and reduces the impact of configuration mistakes. At the same time, the visa service uses global credentials when talking to the authentication service because identity verification is a shared concern, not scoped per domain.

To grant a visa for a specific request, the visa service first identifies the service by verifying its protocol details (address, protocol, port). If the service is bound to a domain, the visa service checks for any `never allow` statements in the domain policy or its restrictions. Then it looks for any `never allow` statements in all parent policy restrictions.

If no `never` statement matches in the delegation chain then the visa service tries to match the request against the domain `allow` statements (in the order as written in ZPL). If an `allow` is found a visa is granted.

6.1. Evaluation Process

Since we allow a domain policy to restrict what services can do when acting as clients, the visa service has two general types of access requests to assess as the following examples illustrate.

Example 1: *A client (not a service) attempts to access a service in domain `Ds`.*

A visa is **denied** if:

1. The policy for domain `Ds` has a matching `never allow` or assertion.
2. The envelope for domain `Ds` has a matching `never allow` or assertion.
3. The envelope for the parent domain of `Ds` (and so on until root) has a matching `never allow` or assertion.

Otherwise a visa is **granted** if:

1. The policy for domain `Ds` has a matching `allow` statement.

Example 2: *A client that is also a service in domain `Dc` attempts to access a service in domain `Ds`.*

A visa is **denied** if:

1. The policy for domain `Ds` has a matching `never allow` or assertion.
2. The envelope for domain `Ds` has a matching `never allow` or assertion.
3. The envelope for the parent domain of `Ds` (and so on until root) has a matching `never allow` or assertion.
4. The policy for domain `Dc` has a matching `never allow` or assertion.
5. The envelope for domain `Dc` has a matching `never allow` or assertion.
6. The envelope for the parent domain of `Dc` (and so on until root) has a matching `never allow` or assertion.

Otherwise a visa is **granted** if:

1. The policy for domain `Ds` has a matching `allow` statement.

7. Enforcement Guarantees

To summarize how domains work:

- A domain owns service names under its DNS suffix.
- A domain policy may define and allow access only to services in that domain.
- Delegator restrictions always constrain delegatee policy.
- Policy rules in a domain only apply to that domain.
- Restrictions compile/evaluate with the delegator's authority, not the delegatee's.
- Attribute visibility is determined by domain credentials and may be narrowed by assertions.

Trusted services play a key role in enforcement by strictly controlling attribute distribution. They must:

- Only return delegated attributes to the entities authorized to receive them.
- Avoid exposing conflicting attribute sets that would place a service under two different delegated domains.

It is important to note that the strict binding of services to delegated domains only works if attributes are correctly managed in the underlying systems.

While assertions give administrators a way to enforce attribute usage within the ZPR ecosystem, ZPR normally relies on external, trusted services to return accurate attribute sets and to avoid exposing attributes that would give a domain administrator influence outside its authorized scope.

When attribute discipline is followed, the delegation model in ZPR achieves three goals simultaneously:

1. It lets large organizations distribute policy authoring to many admins under central control.
2. It keeps the semantics of delegation aligned with the semantics of normal access control through the use of access credentials.
3. It provides clear, auditable boundaries for administrative authority. Misconfigurations can be detected through policy audits, compiler checks, and visa service validation.

8. Example

In this example we consider a company with an accounting department and a marketing department. The company initially deploys ZPR without delegation using the default domain. We first show the monolithic single-domain setup, then show how it decomposes under delegation.

This assumes some familiarity with existing ZPL and introduces some pre-release ZPL features (notably `declare` and `use`).

8.1. Starting point: A single policy

The visa service is configured with the DNS root set to `corp.com` so all the services in ZPL are found at the root (eg, “aboutus.corp.com”).

The ZPL policy looks like this with rules for both accounting and marketing.

```
declare octa-auth as an AuthenticationService
  with
    attr-mapping:"dept -> user.dept",
    attr-mapping:"clearance -> user.clearance"
  provided by
    device.zpr.adapter.cn:"octa1.foo".

declare ldap-svc as an AttributeService
  provided-by
    device.apr.adapter.cn:"ldap.foo".

declare aboutus as a service with port:443
  provided by marketing-service-role:abweb.

declare templates as a service with port:443
  provided by marketing-service-role:templates.

declare mktdb as a service with port:3306
  provided by marketing-service-role:db.

declare timetrack as a service with port:443
  provided by accounting-service-role:timedb.

declare expenselog as a service with port:4343
  provided by accounting-service-role:expenses.

declare audits as a service with port:443
  provided by accounting-service-role:audit.

declare actdb as a service with port:6379
  provided by accounting-service-role:db.


define employee as a user with corp-id:.
define marketing-employee as an employee with dept:marketing.
define fulltimer as an employee with emp-type:fte.
define auditor as an employee with dept:accounting, role:auditor.

allow aboutus to access mktdb.
allow templates to access mktdb.
allow timetrack to access actdb.
allow expenselog to access actdb.
```

```
allow audits to access actdb.
allow marketing-employees to access templates.
allow auditors to access audits.
allow employees to access timetrack.
allow fulltimers to access expenselog.
allow employees to access aboutus.
allow internet-gateway endpoints to access aboutus.
```

8.2. Delegation Administration With Visa Service

The visa service supports delegation through administrative mechanisms all accessed through an API. It manages a set of domains. The first domains must be created by the ZPR administrator but delegated administrators can create domains too if they are permission'd to do so.

The visa service manages its own set of administrators along with what domain they are in and their associated permissions. Domain policies are submitted via the API in ZPL form and the visa service manages the compilation step, ensuring that all the delegation restrictions are applied before accepting/installing the policy. Finally, the visa service provides auditing capabilities, with the correct permissions an auditor can access:

- All the active policy on the ZPRnet and its hierarchical ordering.
- All the users who have access to policy and their access level.
- All historical operations performed on policy and the user access system.

Below is the domain configuration for the monolithic domain. Note that by accessing the attribute service with a "global" credential, it returns the role attributes for both accounting and marketing.

```
{
  "parent_domain":null,
  "domain":"corp.com",
  "validation_services":[
    {
      "service_id": "okta_auth",
      "credential": "global_oktakey.1231299439949"
    }
  ],
  "attribute_services":[
    {
      "service_id": "ldap-svc",
      "credential": "global_apikey.120301230023002"
    },
  ],
  "cnames":[
    {
      "name": "corp.com",
      "value": "aboutus.corp.com"
    }
  ]
```

```
}
```

We now decompose this into two a root domain and two delegated domains: one for marketing and one for accounting.

8.2.1. The Root Policy

This is the policy for the root domain of `corp.com`. All it does is declare two trusted services and some attribute mappings.

```
declare octa-auth as an AuthenticationService
  with
    attr-mapping:"dept -> user.dept",
    attr-mapping:"clearance -> user.clearance"
  provided by
    device.zpr.adapter.cn:"octa1.foo".

declare ldap-svc as an AttributeService
  with
    attr-mapping:"aud -> service.aud",
    attr-mapping:"pubgw -> #endpoint.internet-gateway"
  provided-by
    device.apr.adapter.cn:"ldap.foo".
```

And here is the root domain. The only differences are that we alter the `cname` mapping since the marketing sites will now be in a new domain, and we add the `domain_attribute` configuration section:

```
{
  "parent_domain":null,
  "domain":"corp.com",
  "domain_attribute": {
    "name": "zpr_domain",
    "provider": "okta_auth"
  },
  "validation_services":[
    {
      "service_id": "okta_auth",
      "credential": "global_oktakey.1231299439949"
    }
  ],
  "attribute_services":[
    {
      "service_id": "ldap-svc",
      "credential": "global_apikey.120301230023002"
    },
  ],
  "cnames":[
    {
      "name": "corp.com",
```

```
      "value": "aboutus.marketing.corp.com"
    }
  ]
}
```

8.2.2. Marketing Policy

The marketing ZPR administrator only needs to write policy about services managed by the marketing department.

All the services defined in the marketing domain will be found in DNS in the correct subdomain (which is the domain name, eg, “aboutus.marketing.corp.com”) and no other domain can add or alter names in the marketing namespace.

The marketing ZPL:

```
use octa-auth.corp.com.

use ldap-svc.corp.com with
  attr-mapping:"marketing-service-role -> service.marketing-service-role".


declare aboutus as a service with port:443 and
  provided by marketing-service-role:abweb.

declare templates as a service with port:443
  provided by marketing-service-role:templates.

declare mktdb as a service with port:3306
  provided by marketing-service-role:db.

define employee as a user with corp-id:.
define marketing-employee as an employee with dept:marketing.

allow aboutus to access mktdb.
allow templates to access mktdb.
allow marketing-employees to access templates.

allow employees to access aboutus.
allow internet-gateway endpoints to access aboutus.
```

The marketing domain:

```
{
  "parent_domain":"corp.com",
  "domain":"marketing",
  "validation_services":[
    {
      "service_id": "okta_auth.corp.com",
      "credential": "_inherit"
```

```
    }
  ],
  "attribute_services":[
    {
      "service_id": "ldap-svc.corp.com",
      "credential": "marketing_key.9458488499"
    },
  ],
  "restrictions":[
    "never allow aud:internal services to access endpoint.internet-gateway services"
  ]
}
```

Since `domain` is set to `marketing` and the `parent_domain` is `corp.com`, the DNS root domain for this Visa Service "domain" will be `marketing.corp.com`.

8.2.3. Accounting Policy

The accounting ZPL is similarly the accounting-relevant subset of the monolithic policy.

```
use octa-auth.corp.com.

use ldap-svc.corp.com with
  attr-mapping:"accounting-service-role -> service.accounting-service-role".


declare timetrack as a service with port:443
  provided by accounting-service-role:timedb.

declare expenselog as a service with port:4343
  provided by accounting-service-role:expenses.

declare audits as a service with port:443
  provided by accounting-service-role:audit.

declare actdb as a service with port:6379
  provided by accounting-service-role:db.

define employee as a user with corp-id:.
define fulltimer as an employee with emp-type:fte.
define auditor as an employee with dept:accounting, role:auditor.

allow timetrack to access actdb.
allow expenselog to access actdb.
allow audits to access actdb.

allow auditors to access audits.
allow employees to access timetrack.
allow fulltimers to access expenselog.
```

The accounting domain:

```
{
  "parent_domain":"corp.com",
  "domain":"accounting",
  "validation_services":[
    {
      "service_id": "okta_auth.corp.com",
      "credential": "_inherit"
    }
  ],
  "attribute_services":[
    {
      "service_id": "ldap-svc.corp.com",
      "credential": "accounting_key.5652222345"
    },
  ],
  "restrictions":[
    "never allow aud:internal services to access endpoint.internet-gateway services"
  ]
}
```

Since `domain` is set to `accounting` and the `parent_domain` is `corp.com`, the DNS root domain for this Visa Service "domain" will be `accounting.corp.com`.

8.3. The Policy Management System

The Visa Service manages user access to domain content by keeping a database of user records along with their authentication details and domain permissions. The Visa Service supports a set of roles that can be assigned to users.

- **policy_admin**:
  - Read and write access to all policy in all domains.
  - Read and write access to all domain and user configuration.
- **policy_writer**:
  - Read and write access to policy in all domains.
- **policy_auditor**:
  - Read access to all policy in all domains.
  - Read access to all Visa Service domain and user configuration.
- **policy_reader**:
  - Read access to all policy in all domains.
- **domain_admin**:
  - Read and write access to policy in a domain.
  - Read and write access to user configuration in a domain.
  - Ability to create/edit/delete sub-domains.
- **domain_writer**:
  - Read and write access to policy in a domain.
- **domain_auditor**:
  - Read access to policy in a domain.

  - Read access to user configuration in a domain.
- **domain_reader**:
  - Read access to policy in a domain.

For this example, the ZPR administrator adds the `marketing_editor` and the `accounting_editor` to the visa service user database, each given write access to their domain. Here is the definition of the `marketing_editor`:

```
{
   "user_id": "marketing_editor",
   "provider": "vs.zpr",
   "access": [
      { "role": "domain_writer", "domain": "marketing" }
   ],
   "status": "enabled"
}
```

And the `accounting_editor`:

```
{
   "user_id": "accounting_editor",
   "provider": "vs.zpr",
   "access": [
      { "role": "domain_writer", "domain": "accounting" }
   ],
   "status": "enabled"
}
```

The Visa Service supports external authentication. Obviously this requires that there is already enough policy present for a authentication service to connect, but once one is available, a user record can be inserted using it. For example, here is the `auditor` record which is authenticated by an Okta service.

```
{
   "user_id": "auditor",
   "provider": "okta",
   "issuer": "https://corp.okta-gw.corp.com",
   "subject": "00u1234567890abcdef",
   "access": [
      { "role": "policy_auditor" },
   ],
   "status": "enabled"
}
```

Each domain editor creates their own ZPL and installs it into the visa service using the API. When the domain policies are submitted, the visa service performs compilation and incorporates all the restrictions applied through delegation.

Note that the visa service is not configured with the trusted service credentials directly – those are provided to the domain administrators out of band and submitted with their policy configuration.

9. Revision History

1. Jun 27, 2026
    a. Initial version.
2. Jul 20, 2026
    a. Slightly expanded 1.1, “Why Delegation?”
    b. Rename 1.2.3, “Service Discovery” to “Service Definition”.
    c. Rewrote section 3, “Delegation Attributes”.
3. Jun 27, 2026
    a. Reframe as an RFC for the Reference Implementation.
    b. Replace the abstract admin hierarchy and proof-of-delegation tokens with the **policy domain** as the unit of delegation (envelope vs. contents).
    c. Add the Policy Management System (PMS): REST API, domain sub-delegation, and role-based user access.
    d. Describe Visa Service compilation and evaluation of delegated policy.
    e. Add a worked example decomposing a monolithic policy into root, marketing, and accounting domains.